Architekturspezifikation und Entwickler-Handoff: High-Performance Multi-Stream & Chat-Aggregations-SaaS

1. Zweck, Systemkontext und Normative Korrekturen

Diese technische Spezifikation konsolidiert und erweitert die Grundlagen des Multi-Stream- und Chat-Aggregations-Systems zu einer architektonisch belastbaren Entwickler-Dokumentation gemäß den Standards ISO/IEC/IEEE 42010 und arc42. Das System adressiert die Anforderungen von Erotikmodels und spezialisierten Agenturen, die hochparallele Live-Streams über RTMP- und SRT-Ingest an eine Vielzahl externer Zielportale wie Chaturbate, Stripchat oder MyFreeCams verteilen, eingehende Chat-Kommunikationen sowie Interaktions-Events zentralisieren und automatische Feedback-Schleifen sowie Monetarisierungs-Analysen bereitstellen. Um das Qualitätsniveau des NexusSaaS-Referenzmodells vollständig zu erreichen, unterzieht diese Spezifikation den ursprünglichen Entwurf mehreren normativen Korrekturen.

In der Authentifizierungsschicht wird der veraltete OAuth 2.0 Resource Owner Password Credentials Grant strikt gestrichen. Gemäß den Vorgaben von RFC 9700 (OAuth 2.0 Security Best Current Practice) kommt für moderne Web- und Client-Anwendungen ausschließlich der Authorization Code Grant in Kombination mit PKCE (Proof Key for Code Exchange) oder die Anbindung an einen etablierten OpenID Connect (OIDC) Provider zum Einsatz. Ergänzend wird die Token-Handhabung durch DPoP (Demonstrating Proof-of-Possession) gehärtet, um den Diebstahl und die Wiederverwendung von Access Tokens auf Netzwerkebene wirksam zu unterbinden.

Im Bereich des Datenschutzes und der System-State-Verwaltung wird die technische Erfüllung von Art. 17 DSGVO (Recht auf Löschung) explizit von einfachen Applikations-Soft-Deletes entkoppelt. Ein Soft-Delete stellt lediglich eine Statusanpassung auf Applikationsebene dar und erfüllt für sich genommen keine Löschverpflichtungen. Die Löscharchitektur kombiniert daher eine physische Löschkaskade über operative Datenbankspeicher, Caches, Objektspeicher und Protokolle hinweg mit einer dedizierten Crypto-Shredding-Pipeline für historisierte Backups.

Für die Laufzeit-Isolation gilt, dass das reine Ausführen generierter Skripte oder Connector-Prozesse mittels einfacher Container-Befehle keine ausreichende Sicherheitsgrenze gegenüber dem Host-Betriebssystem bietet, da der Kernel geteilt wird. Die Container-Infrastruktur nutzt daher gehärtete, rootless Runner-Setups ohne Zugriff auf den Docker-Socket des Hosts. Ergänzend kommen Seccomp- und AppArmor-Profile, strikte Limits für Ressourcen und Dateisysteme sowie explizite Egress-Netzwerk-Allowlisten zum Einsatz, um Ausbrüche aus der Ausführungsumgebung zu verhindern.

Hinsichtlich der Zustandsverwaltung wird eine klare Unterscheidung zwischen deterministischen Kontrollflüssen und nicht-deterministischen Ausgaben von externen Live-Stream-Quellen sowie KI-gestützten Auto-Respondern getroffen. Sämtliche Systemzustände werden über zustandsbehaftete Graph-Orchestratoren und dauerhafte PostgreSQL-Checkpointer verwaltet, während flüchtige Interaktionsdaten über performante Event-Busse geführt werden.

2. Architekturentscheidungen (ADRs)

ADR-ID	Architekturentscheidung	Technische Begründung & Systemfolge
ADR-01	Wahl von SRS (Simple Realtime Server) als zentrale Media Ingest & Fan-out Engine	SRS basiert auf einer Coroutine-Architektur (State-Threads), die extrem hohe Parallelität bei minimalem CPU-Overhead pro Verbindung bietet. Es nimmt RTMP- und SRT-Ingest auf und führt transkodierungsfreie RTMP-Re-Push-Operationen zu Zielportalen durch.
ADR-02	Architektonische Trennung stateful WebSocket-Gateways von stateless Anwendungsdiensten	Der Empfang von Zehntausenden WebSocket-Verbindungen bindet dauerhaft Arbeitsspeicher auf Edge-Knoten. Die Entkopplung über einen Pub/Sub-Backplane erlaubt das unabhängige Skalieren von Gateway-Knoten ohne Beeinträchtigung der Geschäftslogik.
ADR-03	Einsatz von Redis Streams als zentraler Inbound Chat Event Bus	Redis Streams bieten geordnete Nachrichtenprotokolle (XADD), entkoppeln den Hochfrequenz-Empfang externer Platform-Connectors von der Auslieferung und erlauben Reconnect-Abfragen über Zeitfenster (XREVRANGE).
ADR-04	Speicherung von Portal-Credentials via KMS Envelope Encryption	Zugangsdaten und Tokens für Drittplattformen werden auf Spaltenebene mittels AES-256-GCM verschlüsselt. Der Data Encryption Key (DEK) wird pro Eintrag generiert und durch einen HSM-geschützten Master Key (KEK) eingewickelt.
ADR-05	Mandantenisolation über PostgreSQL Row Level Security (RLS) mit FORCE RLS	Alle mandantenbezogenen Tabellen erhalten eine tenant_id. RLS wird erzwungen (FORCE ROW LEVEL

ADR-ID	Architekturentscheidung	Technische Begründung & Systemfolge
		SECURITY), damit Tabelleneigentümer die Regeln nicht umgehen. Die Anwendungsrolle besitzt kein BYPASSRLS.
ADR-06	Transaktionale Entkopplung externer Nebenwirkungen mittels Outbox Pattern	Zustandänderungen wie OBS-Szenenwechsel bei Spendenzielen werden transaktional in eine Outbox-Tabelle geschrieben. Asynchrone Worker arbeiten diese ab, um DB-Locks bei Latenzen externer APIs zu verhindern.
ADR-07	Strikte Idempotenz für Inbound Webhooks und Ingest-Events	Eingehende Signale von Zahlungsdienstleistern oder Autorisierungs-Callbacks speichern Event-UUIDs in einer Datenbank-Inbox mit UNIQUE-Constraint, bevor Geschäftslogik ausgeführt wird.
ADR-08	Verankerung von Crypto-Shredding als Primärstrategie für DSGVO Art. 17 in Backups	Durch das unwiderrufliche Löschen des mandantenspezifischen Key Encryption Keys (KEK) im KMS werden sämtliche historisierte Backups und unstrukturierte Log-Dateien des Mandanten augenblicklich unlesbar.

---

3. Zielarchitektur und Systembausteine

Die Gesamtsystemarchitektur ist als cloud-native, verteilt skalierbare Microservices-Landschaft aufgebaut, die eine strikte Trennung zwischen der Hochfrequenz-Medienverarbeitung, dem Echtzeit-Messaging und der persistenten Anwendungslogik durchsetzt.
Die Eintrittsschicht für Medien-Feeds bildet das Ingest & Streaming Gateway, welches auf hochperformanten SRS-Instanzen basiert. Dieses Gateway nimmt eingehende Feeds über RTMP (TCP-Port 1935) oder SRT (UDP-Port 6000) entgegen. Bei jedem Verbindungsaufbau führt das Gateway synchrone HTTP-Callbacks zur API-Schicht aus, um die Autorisierung des Stream-Keys sowie das Einhalten von Tarifgrenzen zu prüfen. Nach erfolgreicher Freigabe verteilt eine dynamische Fan-Out Worker Pipeline den Feed als parallele RTMP-Streams an die externen Zielportale weiter, ohne dass rechenintensive Transkodierungen auf dem Server stattfinden müssen.
Die Chat- und Ereignisverarbeitung erfolgt getrennt über dedizierte Platform Connector

Services. Für jedes aktive Model-Profil halten diese Adapter dauerhafte Inbound-Verbindungen (WebSockets, WSS, REST-Polling oder proprietäre Push-Schnittstellen) zu den Zielportalen aufrecht. Die Adapter transformieren eingehende Nachrichten, Spenden und System-Events in ein einheitliches, typisiertes JSON-Standardformat und übergeben sie an den zentralen Event Bus, der auf einem Redis Cluster betrieben wird.
Für die Auslieferung der aufbereiteten Daten an die Endgeräte der Creator kommt ein entkoppelter WebSocket Gateway Pool zum Einsatz. Diese zustandsbehafteten Edge-Dienste verwalten bis zu 50.000 aktive Client-Verbindungen pro Knoten und synchronisieren sich über das Redis Pub/Sub-Backplane. Dadurch wird sichergestellt, dass jede Statusänderung und jede Chat-Nachricht ohne nennenswerte Verzögerung auf den Bildschirmen der Nutzer erscheint.
Die Anwendungslogik und Persistenz wird durch den Core API Service (aufgebaut auf FastAPI) in Verbindung mit einer PostgreSQL-Datenbank und einer Cloud-KMS-Infrastruktur gesteuert. PostgreSQL speichert Geschäftsobjekte, Benutzerkonten und Systemkonfigurationen unter erzwungener Row-Level-Security. Sämtliche Zugangsdaten für externe Portale verbleiben geschützt in einem zentralen KMS Envelope Encryption Vault und werden erst im Arbeitsspeicher der Connector-Dienste für den Verbindungsaufbau entschlüsselt.
Der Datenfluss innerhalb dieses Gefüges ist strikt linear und fehlertolerant ausgelegt: Der Client sendet ein Medien-Feed an das Ingest Gateway, welches über die API autorisiert wird und den Fan-Out startet. Parallel verbinden sich die Chat Connectors mit den Zielportalen, speisen Events in Redis Streams ein, woraufhin die WebSocket-Gateways die Nachrichten an die Benutzeroberfläche weiterleiten. Bei Ausfall einzelner Komponenten puffert der Redis Event Bus ausstehende Nachrichten, sodass keine Interaktion verloren geht.

4. Streaming- und Fan-Out-Architektur

4.1 Ingest-Protokollvergleich und Laufzeitverhalten

Die Plattform unterstützt RTMP und SRT als primäre Ingest-Protokolle, die sich hinsichtlich ihres Übertragungs- und Latenzverhaltens im Live-Betrieb grundlegend unterscheiden. RTMP basiert auf TCP und garantiert die vollständige Paketübertragung in der exakten Reihenfolge. Bei schwankenden Netzwerkverbindungen führt dieses Head-of-Line-Blocking jedoch zu Retransmissions, was Pufferzeiten verlängert und die Latenz spürbar erhöht.
SRT hingegen setzt auf UDP auf und integriert einen frei konfigurierbaren ARQ-Fehlerkorrektur-Layer (Automatic Repeat reQuest). Dies ermöglicht eine extrem niedrige Latenz selbst über instabile, drahtlose Verbindungen oder bei mobilen Streaming-Setups.

Streaming-Parameter	RTMP Ingest Pipeline	SRT Ingest Pipeline
Transportprotokoll	TCP (Standard-Port 1935)	UDP (Konfigurierbar, Standard-Port 6000)
Latenzbereich	2.000 ms – 5.000 ms (abängig von TCP-Buffer)	200 ms – 1.000 ms (einstellbares ARQ-Zeitfenster)
Fehlerkorrektur-Mechanismus	TCP In-Order Retransmission (Verbindungsabbruch bei Verlust)	Selective ARQ / UDP-Packet recovery & AES-Verschlüsselung
Typischer Einsatzzweck	Festnetz-Studio-Setups via OBS oder vMix	Mobile Setups ("On-the-Road"), instabile Heimnetze
Target Forwarding Format	Direkter RTMP Pass-Through zu Cam-Portalen	SRT-Inbound zu RTMP-Outbound Multiplexing

4.2 SRS Engine, Multi-Core Skalierung und Fast Playout

SRS wird als Ingest- und Fan-Out-Engine betrieben. Um trotz des Single-Threaded State-Threads (ST) Coroutine-Modells von SRS moderne Server-Hardware voll auszulasten, kommt eine prozessbasierte Multi-Core-Architektur zum Einsatz:

- **Multi-Prozess Scaling via Socket Reuse**: Auf Betriebssystemebene wird die Socket-Option SO_REUSEPORT genutzt. Dies erlaubt es mehreren unabhängigen SRS-Prozessinstanzen, auf demselben Ingest-Port zu lauschen. Der Linux-Kernel verteilt eingehende TCP- und UDP-Verbindungen automatisch gleichmäßig auf die verfügbaren CPU-Cores.
- **Dynamisches Routing über HTTP-Callbacks**: Erreicht ein neuer Stream das Gateway, setzt SRS einen synchronen HTTP-HOOK-Request (on_publish) an die Core-API ab. Die API verifiziert den Stream-Key, schlägt die hinterlegten Portal-Credentials in der Datenbank nach und liefert eine dynamische Liste von Ziel-RTMP-URIs zurück. SRS startet daraufhin für jedes Ziel eine interne Forwarder-Coroutine, die den Datenstrom spiegelt.
- **GOP-Cache und Playout-Optimierung**: SRS hält das jeweils letzte Group of Pictures (GOP) im Arbeitsspeicher vor. Sobald die Verbindung zu einem externen Portal nach einem Netzwerkwackler neu aufgebaut wird, übermittelt SRS sofort einen gültigen I-Frame. Dies verhindert Verpixelungen oder Verwerfungen auf den Zielplattformen und verkürzt die Playout-Startzeit erheblich.

5. Chat-Aggregations- und Messaging-Architektur

5.1 Platform Connectors und Event-Normalisierung

Die heterogene Landschaft externer Cam-Portale nutzt unterschiedliche Verbindungsprotokolle, die von Standard-WebSockets über Pusher-Services bis hin zu proprietärem REST-Polling reichen. Die Platform Connector Services kapseln diese Komplexität, indem sie für jeden aktiven Kanal einen isolierten Client ausführen. Eingehende Datenpakete werden unmittelbar nach Empfang in ein einheitliches, typisiertes JSON-Event-Schema transformiert, welches sowohl textuelle Nachrichten als auch finanzielle Interaktionen präzise strukturiert:

```
{
  "event_id": "evt_9876543210_abc",
  "tenant_id":
"c0a80101-8[span_101](start_span)[span_101](end_span)[span_105](start_
span)[span_105](end_span)c0b-11ef-9000-0242ac120002",
  "stream_id": "str_1122334455",
  "platform": "stripchat",
  "event_type": "tip",
  "timestamp":
"[span_102](start_span)[span_102](end_span)[span_106](start_span)[span
_106](end_span)2026-11-09T14:20:00.123Z",
  "sender": {
    "platform_user_id": "usr_998877",
    "username": "VipDonor99",
    "badge_level": "gold"
```

```
  },
  "payload": {
    "message": "Great show!",
    "tokens_spent": 500,
    "fiat_equivalent_usd": 25.00,
    "is_private_request": false
  }
}
```

5.2 Event Bus und Persistence Buffer via Redis Streams

Die normalisierten Chat-Ereignisse werden über den Befehl XADD sofort in mandantenspezifische Redis Streams geschrieben (stream:chat:{tenant_id}). Die Architektur nutzt Redis Streams als Entkopplungsschicht zwischen dem unvorhersehbaren Eintreffen externer Nachrichten und der Zustellung an die Endnutzer.
Über Consumer Groups lesen verschiedene nachgelagerte Dienste dieselbe Nachricht ab: Die WebSocket-Gateways leiten das Event an den Client weiter, während ein asynchroner Analytics-Worker das Event zur aggregierten Umsatzstatistik in PostgreSQL verbucht. Trennt ein Client vorübergehend die Verbindung, kann die Benutzeroberfläche nach dem Wiederaufbau über den Befehl XREVRANGE exakt diejenigen Nachrichten nachfordern, die während der Offline-Phase im Stream aufgelaufen sind, ohne die SQL-Datenbank zu belasten.

5.3 WebSocket Pool Scaling und Thundering Herd Prevention

Um Zehntausende parallele Clients stabil zu bedienen, wird die WebSocket-Auslieferungsschicht als verteilter, zustandsbehafteter Node-Pool betrieben. Jede Gateway-Instanz führt eine lokale Zuordnungstabelle im Arbeitsspeicher, die festhält, welche Mandanten-Sessions aktuell auf diesem spezifischen Server gehalten werden.
Trifft eine Nachricht über das Redis Pub/Sub-Backplane auf einem Gateway-Knoten ein, prüft dieser augenblicklich seine Arbeitsspeicher-Tabelle und stellt die Nachricht nur an diejenigen Socket-Verbindungen zu, die lokal aktiv sind. Ein integrierter Heartbeat-Sweep sendet im Abstand von 30 Sekunden Ping-Frames an alle offenen Sockets. Antworte ein Client innerhalb von 10 Sekunden nicht mit einem Pong-Frame, wird die TCP-Verbindung serverseitig getrennt und der zugewiesene Speicher freigegeben.
Bei der Aktualisierung oder Skalierung von Gateway-Instanzen schützt ein gestaffeltes Entlastungsverfahren (Graceful Drain Procedure) das Gesamtsystem vor Überlastungsschocks. Der betroffene Server wird zunächst aus der Routing-Tabelle des Load-Balancers entfernt, sodass keine neuen Handshakes mehr zugewiesen werden.
Anschließend sendet der Server über ein einstellbares Zeitfenster hinweg schrittweise Schließsignale mit dem Reconnect-Statuscode an die verbundenen Clients. Durch die Verteilung dieser Schließbefehle auf beispielsweise 1.000 Trennungen pro Sekunde wird verhindert, dass zehntausende Clients zeitgleich versuchen, eine neue Verbindung aufzubauen (Thundering-Herd-Effekt), was nachgelagerte Auth-Dienste überlasten würde.

6. Credential-Sicherheit und Envelope-Encryption

6.1 Bedrohungsmodell und Schutzziele

Die Speicherung von Authentifizierungs-Tokens, API-Schlüsseln und Passwort-Credentials für externe Streaming-Portale stellt eines der kritischsten Sicherheitsrisika der Systemarchitektur dar. Ein Kompromittieren dieser Daten würde Dritten die vollständige Kontrolle über die Portale der Models sowie deren Einnahmen ermöglichen. Eine reine Dateisystem- oder Festplattenverschlüsselung schützt das System lediglich gegen den physischen Verlust von Datenträgern, bietet jedoch keinen Schutz, wenn ein Angreifer Lesezugriff auf die Datenbank erlangt oder ein Prozess mit Lese-Rechten kompromittiert wird.

6.2 Kryptographische Hierarchie und Data Encryption Keys

Um diesen Risiken zu begegnen, wird die Speicherung sämmtlicher sensitiver Zugangsdaten über ein zweistufiges KMS Envelope Encryption Verfahren abgesichert:
Ein zentraler Key Encryption Key (KEK) verbleibt dauerhaft in einem dedizierten, hardwaregeschützten Sicherheitsmodul (HSM) des Cloud-KMS-Anbieters und verlässt dieses zu keinem Zeitpunkt im Klartext. Für jeden einzelnen Schreibvorgang von Zugangsdaten generiert die Anwendung über die KMS-API einen neuen, eindeutigen 256-Bit Data Encryption Key (DEK).
Die Verschlüsselung des Plaintext-Credentials erfolgt direkt im Arbeitsspeicher des Dienstes unter Verwendung des AES-256-GCM-Algorithmus. Bei dieser Operation wird zusätzlich eine kryptographisch authentifizierte Zeichenkette (Additional Authenticated Data / Encryption Context) fest eingebunden, die sich aus der tenant_id und der spezifischen platform_id zusammensetzt. Sollte ein Angreifer versuchen, einen verschlüsselten Datensatz in die Datenbankzeile eines anderen Mandanten zu kopieren, schlägt die Authentizitätsprüfung der GCM-Tag-Verifizierung beim Entschlüsseln fehl.
Nach der Ausführung des Verschlüsselungsbefehls wird der Plaintext-DEK unverzüglich aus dem Hauptspeicher der Anwendung gelöscht und mit null überschrieben (zeroised). In der Datenbank werden ausschließlich das erzeugte Ciphertext-Blob, der verschlüsselte DEK (Wrapped DEK), der Initialisierungsvektor (IV), der GCM-Authentication-Tag sowie die KEK-Versionsnummer abgesichert abgelegt.
Zur Wiederherstellung der Zugangsdaten sendet der autorisierte Connector-Prozess den Wrapped DEK an das KMS zurück, welches diesen unter Einhaltung strikter IAM-Richtlinien entschlüsselt und den kurzlebigen Plaintext-DEK über eine gesicherte TLS-Verbindung an die Anwendung übergibt. Die Anwendung entschlüsselt den Datensatz, baut die Verbindung zum Zielportal auf und löscht den DEK umgehend wieder aus dem Speicher.

7. Multi-Tenancy und Datenbank-Design

7.1 Datenmodell und Isolation

Das Datenbankdesign modelliert die Mandantenfähigkeit als Kerneigenschaft über sämtliche Entitäten hinweg. Ein Mandant (tenants) bildet die oberste organisatorische Einheit für einzelne Models oder Agenturen.

```
-- Kern-Schema für Mandantenverwaltung und strikte RLS-Isolation

CREATE TABLE tenants (
```

```
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    kms_kek_arn VARCHAR(1024) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp()
);

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash
VARCHAR([span_68](start_span)[span_68](end_span)[span_76](start_span)[
span_76](end_span)255) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp()
);

CREATE TABLE tenant_memberships (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role VARCHAR(64) NOT NULL DEFAULT 'member',
    UNIQUE (tenant_id, user_id)
);

CREATE TABLE platform_accounts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    platform_name VARCHAR(64) NOT NULL,
    account_username VARCHAR(255) NOT NULL,
    encrypted_credentials BYTEA NOT NULL,
    wrapped_dek BYTEA NOT NULL,
    iv BYTEA NOT NULL,
    auth_tag BYTEA NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp()
);

-- Aktivierung und Erzwingung von Row Level Security (RLS)
ALTER TABLE platform_accounts ENABLE ROW LEVEL SECURITY;
ALTER TABLE platform_accounts FORCE ROW LEVEL SECURITY;

-- Sicherheits-Policy definieren
CREATE POLICY tenant_isolation_platform_accounts ON platform_accounts
    FOR ALL
    TO application_role
    USING (tenant_id = NULLIF(current_setting('app.tenant_id', true),
'')::uuid)
    WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id',
true), '')::uuid);
```

7.2 Implementierung der PostgreSQL Row-Level Security

Die Mandantentrennung stützt sich auf PostgreSQL Row-Level Security, um Cross-Tenant-Datenlecks auf Datenbankebene unmöglich zu machen. Die RLS-Policy nutzt die Sitzungsvariable app.tenant_id, die bei jeder Transaktion lokal gesetzt wird. Durch die Anwendung der Anweisung FORCE ROW LEVEL SECURITY wird sichergestellt, dass die Regeln auch für den Eigentümer der Tabelle greifen. Die Anwendungsrolle application_role besitzt keinerlei Administratorrechte und ist nicht mit dem Privileg BYPASSRLS ausgestattet. Beim Ausführen von Abfragen setzt der Datenbank-Treiber zu Beginn jeder Transaktion die Anweisung SET LOCAL app.tenant_id = '...' ab. Dadurch schränkt PostgreSQL das Abfrage-Ergebnis automatisch auf diejenigen Zeilen ein, deren tenant_id exakt mit dem gesetzten Wert übereinstimmt. Wird im Anwendungscode versehentlich eine WHERE-Klausel vergessen, garantiert die Datenbank, dass niemals Daten fremder Mandanten zurückgeliefert oder überschrieben werden.

8. Authentifizierung, RBAC und API-Spezifikation

8.1 Authentifizierungsstruktur und OAuth 2.0 PKCE Flow

Gemäß den Sicherheitsvorgaben von RFC 9700 nutzt die Schnittstelle für Client-Anwendungen den OAuth 2.0 Authorization Code Grant in Kombination mit PKCE. Nutzer authentifizieren sich gegen ein OIDC-konformes Identity-System, welches nach erfolgreicher Prüfung einen kurzlebigen Access Token (Gültigkeit 15 Minuten) sowie einen kryptographisch geschützten Refresh Token ausstellt.
Refresh Tokens unterliegen einer strikten Rotation: Bei jeder Verwendung zur Token-Erneuerung wird der alte Refresh Token entwertet und durch einen neuen ersetzt. Wird ein bereits benutzter Refresh Token erneut präsentiert, erkennt das System den Replay-Versuch sofort und widerruft die gesamte Token-Familie des Benutzers.

8.2 API-Endpunkt-Spezifikation

Methode	Endpunkt	Zweck / Beschreibung	RBAC-Rolle	Isolation & Security Gate
POST	/api/v1/auth/token	Token-Exchange (Auth Code + PKCE)	Anonym	PKCE Verification, Rate Limiting
POST	/api/v1/streams/verify	SRS Ingest-Autorisierungs-Callback	Engine / System	Shared Secret, IP-Allowlist
GET	/api/v1/streams/target[span_116](start_span)[span_116](end_span)s	Abfragen aktiver Restream-Ziele	Creator / Admin	Auth-Token + RLS Context (tenant_id)
POST	/api/v1/credentials	Hinterlegen von Portal-Zugangsdat	Creator / Admin	Auth-Token, KMS Envelope

Methode	Endpunkt	Zweck / Beschreibung	RBAC-Rolle	Isolation & Security Gate
		en		Encryption
WS	/ws/v1/chat[span_172](start_span)[span_172](end_span)[span_176](start_span)[span_176](end_span)	Echtzeit Chat Aggregation Feed	Creator / Admin	Ticket-Auth via Query Token, WSS Encryption
POST	/api/v1/outbox/trigger	Auslösen von OBS-Webhooks / Szenen	System / Worker	Outbox Verification, HMAC Signatur
POST	/api/v1/webhooks/stripe	Verarbeitung von Inbound-Abo-Zahlungen	Extern (Stripe)	HMAC Signaturprüfung, Unique Inbox
DELETE	/api/v1/tenants/me	Einleiten der Löschung nach DSGVO Art. 17	Tenant Owner	Re-Authentifizierung, Crypto-Shredding Trigger

9. Webhooks, Queues und asynchrone Prozesse

9.1 Stripe Inbound Webhooks und Idempotenz-Inbox

Das Verarbeiten externer Webhooks (beispielsweise von Stripe für Abonnement-Updates) erfolgt über eine idempotente Inbox-Architektur, um doppelte Buchungen bei wiederholter Zustellung auszuschließen. Eingehende HTTP-POST-Requests treffen am Fast-Path-Handler der API ein, welcher als Erstes die kryptographische HMAC-Signatur über den Rohdaten-Body verifiziert.
Nach erfolgreicher Verifizierung schreibt der Handler die eindeutige event_id des Webhooks zusammen mit dem Empfangszeitstempel in die Datenbanktabelle webhook_inbox. Diese Tabelle besitzt eine UNIQUE-Restriktion auf der Spalte event_id.
Sollte Stripe dasselbe Ereignis aufgrund eines Netzwerkfehlers erneut senden, schlägt der Datenbank-Insert wegen der UNIQUE-Restriktion fehl. Die Anwendungslogik fängt diesen Zustand ab, bricht die Transaktion ohne Fehler ab und liefert sofort den Status-Code HTTP 200 OK an den Aufrufer zurück. Die eigentliche Verarbeitung der Geschäftsvorfälle findet zeitlich entkoppelt im Hintergrund statt: Ein Inbox-Worker liest ungelesene Datensätze aus, führt die Zustandsänderung (z. B. Hochstufen des Benutzer-Limits) in einer isolierten Transaktion durch und markiert das Event als verarbeitet.

9.2 Transactional Outbox Pattern für externe Auswirkungen

Wenn ein Chat-Event oder eine Erreichung eines Spendenziels eine externe Aktion auslösen muss – wie beispielsweise das Senden eines Steuer-Webhooks an die lokale Streaming-Software (OBS) des Models –, darf diese Aktion nicht direkt im synchronen Nachrichten-Pfad ausgeführt werden. Externe Aufrufe können durch Netzwerklatenzen blockiert werden oder fehlschlagen, was zu blockierten Datenbanktransaktionen führen würde.
Das System löst diese Kopplung über das Transactional Outbox Pattern: Zusammen mit der

primären Zustandsänderung schreibt die Anwendung den auszuführenden externen Befehl in eine Datenbanktabelle namens transactional_outbox. Die Schreiboperation erfolgt innerhalb derselben atomaren Transaktion wie das Business-Event.
Ein separater Outbox-Worker fragt die Tabelle periodisch ab, liest ausstehende Einträge mit der Anweisung FOR UPDATE SKIP LOCKED aus und sendet die Anfragen an die Ziel-APIs. Tritt ein temporärer Netzwerkfehler auf, wendet der Worker eine Exponential-Backoff-Strategie mit zufälliger Abweichung (Jitter) an. Erst nach erfolgreicher Bestätigung durch das Zielsystem wird der Eintrag in der Outbox als erledigt markiert oder gelöscht.

10. Rate-Limiting, Kostenkontrolle und Quota-Management

10.1 Redis Token-Bucket Algorithmus

Zum Schutz der Schnittstellen gegen DoS-Angriffe sowie zur Durchsetzung von Tarifeinschränkungen verwendet das System einen Redis-LUA-basierten Token-Bucket-Algorithmus. Der Algorithmus arbeitet atomar innerhalb von Redis, um Rassebedingungen (Race Conditions) bei parallelen Anfragen zu verhindern. Das Rate-Limiting wird nach verschiedenen Endpunkt-Klassen gestaffelt angewendet:
Für interaktive REST-API-Routen gilt ein Limit von 60 Anfragen pro Minute pro authentifiziertem Nutzer. Eingehende Chat-Events an den Platform Connectors dürfen bis zu 1.000 Events pro Minute pro Kanal verarbeiten, um Spenden-Peaks abzufangen. Für die WebSocket-Auslieferung an die Benutzeroberfläche liegt die Obergrenze bei 500 Nachrichten pro Sekunde pro Verbindung. Wird eines dieser Limits überschritten, verweigert das Gateway die Ausführung, gibt den Status-Code HTTP 429 Too Many Requests zurück und sendet den Header Retry-After mit der verbleibenden Wartezeit in Sekunden an den Client.

10.2 Stream- und Egress-Quotas pro Mandant

Zur Absicherung gegen unkontrollierte Infrastrukturkosten durch verwaiste oder dauerhafte Streaming-Sessions setzt das System strikte Kontingentgrenzen durch:

- **Maximal parallele Ingest-Feeds**: Ein Ingest-Feed im Standard-Tarif, maximal drei parallele Ingest-Feeds im Agentur-Tarif.
- **Maximal parallele Zielportale**: Bis zu drei simultane RTMP-Outbound-Streams im Standard-Tarif, bis zu zehn simultane Zielportale im Pro-Tarif.
- **Maximale Stream-Laufzeit (Wall-Time Cap)**: Ein kontinuierlicher Stream wird nach spätestens 24 Stunden Laufzeit automatisch serverseitig getrennt. Der Creator muss den Feed aktiv neu starten, was verhindert, dass unbeabsichtigt aufrechterhaltene Streams dauerhaft Bandbreite belegen.

11. DSGVO, Privacy by Design und Crypto-Shredding

11.1 Datensparsamkeit und Retention-Schedules

Die Verarbeitung von Nutzerdaten und Chat-Protokollen im Umfeld von Erotik-Plattformen erfordert die konsequente Einhaltung der Grundsätze der Datensparsamkeit und der

Zweckbindung gemäß DSGVO.

Datenkategorie	Verarbeitungszweck	Speicherort	Aufbewahrungsfrist (Retention)
Chat-Protokolle	Aggregation, Auto-Responder, Historie	Redis Streams / PostgreSQL	Physische Löschung nach 30 Tagen
Umsatz- & Tipp-Logs	Abrechnung, Performance-Dashboard	PostgreSQL (Spaltenverschlüsselt)	Gesetzliche Aufbewahrung (10 Jahre)
Portal-Credentials	Automatisierter Verbindungsaufbau	PostgreSQL (Envelope Encrypted)	Sofortige Löschung bei Konto-Trennungs-Request
Flüchtige Zustände	Online-Status, Tipp-Indikatoren	Redis In-Memory Store	Automatische Löschung nach max. 60 Sekunden

11.2 Crypto-Shredding Pipeline für unveränderliche Backups

Die Durchsetzung des Rechts auf Löschung nach Art. 17 DSGVO stellt bei unveränderlichen, verschlüsselten Datenbanksicherungen (Immutable Backups) eine technische Herausforderung dar, da einzelne Datensätze nicht nachträglich aus einer bestehenden Backup-Datei entfernt werden können, ohne deren Integrität zu zerstören. Das System löst diesen Zielkonflikt durch ein durchgängiges Crypto-Shredding-Verfahren.
Reicht ein Nutzer eine Löschanforderung ein, führt das System zunächst die physische Löschung aller zugehörigen Datensätze aus den aktiven Primärdatenbanken, Caches und Objektspeichern durch. Um die in historisierten Datenbanksicherungen verbleibenden Ciphertext-Fragmente ebenfalls unzugänglich zu machen, setzt das System einen gezielten KMS-Befehl ab, der den spezifischen Key Encryption Key (KEK) des betreffenden Mandanten im Hardware-Sicherheitsmodul unwiderruflich löscht.
Ohne diesen KEK ist es mathematisch unmöglich, die mit den zugehörigen Data Encryption Keys (DEKs) verschlüsselten Datenblöcke in den Backups zu entschlüsseln. Die Daten werden dadurch augenblicklich und dauerhaft anonymisiert beziehungsweise vernichtet, ohne dass Schreiboperationen auf den eigentlichen Backup-Medien ausgeführt werden müssen.

12. Nichtfunktionale Anforderungen (NFRs)

NFR-Kategorie	MVP-Akzeptanzkriterium	Architekturelle Durchsetzung / Maßnahme
API-Latenz	P95 < 200 \text{ ms} für synchron beantwortete REST-Routen.	Zustandslose FastAPI-Services, Redis-In-Memory-Caching für Lesezugriffe.
Chat-Zustellung	P99 < 100 \text{ ms} von Empfang am Connector bis zur UI-Anzeige.	Direktes Routing von Redis Streams zu den stateful WebSocket-Gateways.
Fan-Out Latenz	End-to-End Latenz-Zuwachs < 200 \text{ ms} durch das Gateway.	Transkodierungsfreies Pass-Through Forwarding via SRS State-Threads Engine.

NFR-Kategorie	MVP-Akzeptanzkriterium	Architekturelle Durchsetzung / Maßnahme
Verbindungsdichte	Mindestens 50.000 parallele WSS-Verbindungen pro Gateway-Knoten.	Asynchroner Gateway-Layer (Go/Rust), Kernel-Netzwerk-Tuning (somax[span_86](start_span)[span_86](end_span)conn).
Isolation	Null Toleranz für Cross-Tenant-Zugriffe in automatisierten Tests.	PostgreSQL FORCE RLS, dynamische Sitzungsvariablen SET LOCAL ap[span_87](start_span)[span_87](end_span)p.tenant_id.
Disaster Recovery	\text{RPO[span_152](start_span)[span_152](end_span)} < 1 \text{ Minute}, \text{RTO} < 15 \text{ Minuten} für das Kernsystem.	WAL-G-basiertes Continuous Archiving, PostgreSQL Point-in-Time-Recovery.

13. Test-, Resilienzen- und Freigabestrategie

Die Qualitätssicherung ruht auf mehreren Teststufen, die im Rahmen der Continuous-Integration-Pipeline automatisch durchlaufen werden müssen:
Auf der Unit-Test-Ebene werden die Nachrichten-Normalisierer, die Token-Bucket-Logik, die Event-Schemas sowie die Autorisierungs-Policies isoliert auf ihre korrekte Funktion geprüft.
Die Integrationstests verifizieren insbesondere das korrekte Zusammenspiel der Datenbank und der Sicherheitsregeln. Hierbei wird dediziert getestet, ob die PostgreSQL Row-Level Security unter der eingeschränkten Anwendungsrolle application_role zuverlässig greift. Die Testsuite stellt sicher, dass Abfragen ohne vorheriges Setzen der Sitzungsvariable app.tenant_id eine leere Ergebnismenge zurückliefern und Versuche, fremde UUIDs abzufragen, von der Datenbank blockiert werden.
Zur Überprüfung der System-Resilienz werden automatisierte Chaos-Testing-Szenarien durchgeführt. Dazu gehört das gezielte Beenden von SRS-Streaming-Prozessen unter Volllast, um zu verifizieren, dass eingehende Encoder-Feeds automatisch auf verbleibende Edge-Knoten ausweichen und die Fan-Out-Forwarder die Verbindungen zu den Portalen selbstständig neu aufbauen.
Zusätzlich wird das Verhalten bei Netzwerktrennungen im WebSocket-Pool simuliert, um nachzuweisen, dass das gestaffelte Entlastungsverfahren (Graceful Drain) ein Kaskadieren von Fehlern verhindert.
Vor jedem Produktions-Release müssen sämtliche statischen Code-Analysen (Semgrep, Bandit), Secret-Scanner und Abhängigkeits-Überprüfungen ohne Befund für kritische oder hohe Sicherheitsrisiken abschließen. Zusätzlich werden automatisierte Penetrationstests auf API-Ebene ausgeführt, um Schwachstellen bezüglich unterbrochener Objekt-Autorisierungen (BOLA/IDOR) auszuschließen.

14. Umsetzungsplan und Roadmap

Phase	Umfang & Modul	Abnahmekriterien & Outbound-Validation
Phase 0	Architekturentscheidungen, Threat Model & KMS-Design.	ADRs akzeptiert; CI/CD-Pipeline mit statischen Scannern einsatzbereit.
Phase 1	Plattform-Grundlage, Datenmodell & RLS-Policies.	PostgreSQL Schemas deployt; Cross-Tenant-Isolationstests zu 100 % grün.
Phase 2	Identity, Auth (PKCE) & Credential Vault Integration.	OIDC Flow funktionsfähig; KMS Envelope Encryption Unit-Tests erfolgreich.
Phase 3	SRS Media Ingest Gateway & Dynamic Fan-Out Pipeline.	Stabil gemessener Multi-Push von 1 Ingest-Feed auf 3 Test-Portale ohne Frame-Loss.
Phase 4	Platform Chat Connectors & Redis Streams Event Bus.	Nachrichten von 3 Cam-Portalen werden korrekt in das Standard-JSON normalisiert.
Phase 5	Stateful WebSocket Gateway Cluster & Edge Push Engine.	Lasttest mit 50.000 parallelen WebSocket-Verbindungen erfüllt alle NFR-Vorgaben.
Phase 6	Billing, Stripe Webhooks, Quota Controls & Crypto-Shredding.	Idempotente Webhook-Verarbeitung nachgewiesen; Key-Deletion-Drills grün.
Phase 7	Closed Beta Pilotbetrieb mit ausgewählten Test-Models.	Erfolgreicher 14-Tage-Testbetrieb ohne Sicherheits- oder Isolations-Incidents.

15. Repository-Struktur und Audit-Vergleich

Die Ordnerstruktur des Quellcode-Repositorys spiegelt die klare Abgrenzung der Systembausteine wider und verhindert zyklische Abhängigkeiten zwischen den Modulen: multistream-saas/ ├── apps/ │ ├── api/ # FastAPI Core Service (REST, Auth, Ingest Callbacks) │ ├── websocket-gateway/ # High-Performance Stateful WS Pool (Go / Rust) │ ├── chat-connectors/ # Modular Platform Connectors (Stripchat, Chaturbate, etc.) │ └── outbox-worker/ # Background Worker for Retries & External Side-Effects ├── packages/ │ ├── common-domain/ # Shared Types, Entities, Domain Events │ ├── crypto-vault/ # KMS Envelope Encryption & Key Lifecycle Management │ ├── schema-normalizer/ # Unified Chat JSON Schema & Validators │ └── storage/ # PostgreSQL Client (RLS Wrappers) & Redis Stream Drivers ├── migrations/ # Alembic Database Migrations & RLS Policy Definitions ├── infra/ # Terraform / IaC Templates for SRS Cluster, KMS, Helm ├── tests/ │ ├── unit/ # Unit Tests for Rules and Schemas │ ├── integration/ # RLS Isolation & Connection Pool Tests │ └── load/ # k6 / Locust Scripts for WebSocket & Ingest Load Tests ├── pyproject.toml # Pinned Python Dependencies └── compose.yaml # Isolated Local Development Environment Im direkten architektonischen Vergleich mit dem vorausgegangenen Blueprint schließt dieses Dokument sämtliche Sicherheits- und Skalierungslücken vollständig:

Durch die Streichung des veralteten OAuth2 Password Grants zugunsten von Authorization Code + PKCE sowie DPoP wird die Authentifizierungsschicht auf den aktuellen Stand der Technik gemäß RFC 9700 gehoben. Die Einführung von KMS Envelope Encryption stellt sicher, dass sensible Portal-Passwörter und Tokens selbst bei einem vollständigen Datenbank-Dump nicht im Klartext ausgelesen werden können.
Auf der Streaming- und Messaging-Ebene ersetzt die SRS-basierte Multi-Prozess-Architektur in Kombination mit dem Redis-Streams-Event-Bus unzureichende monolithische Ansätze. Das Gesamtsystem ist in der Lage, Hunderte paralleler Video-Feeds verzugsnachfrei zu spiegeln und extrem hohe Chat-Nachrichtenvolumina ohne Datenverlust zu aggregieren.
Zusammen mit der strikten, datenbankseitig erzwungenen Mandantentrennung via PostgreSQL Row-Level Security und der datenschutzkonformen Crypto-Shredding-Pipeline liegt nun ein vollständig ausgearbeiteter Blueprint vor, der als direkte Handlungsgrundlage für Senior Engineers dient.

Quellenangaben

1. Building a Scalable Chat App Using WebSockets, Redis, Kafka, and, https://medium.com/@indraneelsarode22neel/building-a-scalable-real-time-chat-app-from-single-server-to-distributed-powerhouse-614bb3391fa8 2. Ingest and publish an SRT stream with Wowza Streaming Engine, https://www.wowza.com/docs/ingest-and-publish-an-srt-stream-with-wowza-streaming-engine 3. AI Agent Security and Governance - Deployments.ai, https://deployments.ai/security 4. How to Store OAuth Tokens Securely for AI Apps - Agentic Fabriq, https://www.agenticfabriq.com/blog/store-oauth-tokens-securely 5. Envelope Encryption and Key Management: KMS, HSM, Key Rotati, https://iotdigitaltwinplm.com/envelope-encryption-kms-hsm-key-management-architecture-key-rotation-2026/ 6. Chat Application Architecture, Explained - GetStream.io, https://getstream.io/blog/chat-application-architecture/ 7. Why we put chat messages in Redis streams (and plan to move old, https://www.reddit.com/r/cpp/comments/1u13p31/why_we_put_chat_messages_in_redis_streams_and/ 8. Best Iptv Streaming Server Software | 2026 Expert Picks - Gitnux, https://gitnux.org/best/iptv-streaming-server-software/ 9. SRS is a simple, high-performance, AI-driven real-time media server, https://github.com/ossrs/srs 10. SRS (Simple Realtime Server) | SRS, https://ossrs.io/lts/en-us/ 11. Build Your Own Video Streaming Server (RTMP Server) - WpStream, https://wpstream.net/streaming-server/ 12. GitHub - im-pingo/liveforge: High-performance multi-protocol live, https://github.com/im-pingo/liveforge 13. Best VPS for Streaming in 2026: Bitrate, Bandwidth & Setup Guide, https://x-zoneservers.com/blog/best-vps-for-streaming-in-2026-bitrate-bandwidth-and-setup-guide 14. How to Build a Video Streaming App: 2026 Architecture Guide, https://www.forasoft.com/blog/article/custom-video-streaming-app-guide 15. Scalable Multi-instance WebSockets with Redis Pub/Sub, https://www.superdevacademy.com/en/blogs/golang-multi-instance-websocket-redis-pubsub 16. WebSocket Architecture at Scale: Millions of Connections - ScaleDojo, https://scaledojo.dev/blogs/websocket-architectures-at-scale-handling-millions-of-persistent-connections 17. Scaling Real-Time WebSockets to 50k Connections with Redis, https://codelifeai.com/blog/scaling-websockets-redis-streams 18. SRS - Load Balancing Streaming Servers, https://ossrs.io/lts/en-us/blog/load-balancing-streaming-servers 19. Redis and WebSockets: Managing Real-Time Web Interfaces,

https://reintech.io/blog/redis-websockets-real-time-web-interfaces 20. SRS - State Threads for Internet Applications, https://ossrs.io/lts/en-us/blog/state-threads-for-internet-applications 21. SRS 10th Anniversary and 2023 Year-end Review, https://ossrs.io/lts/en-us/blog/srs-ten-years 22. Video — list of Rust libraries/crates // Lib.rs, https://lib.rs/multimedia/video 23. Vault and keys - Altostrat Documentation, https://altostrat.io/docs/studio/en/ai-safety/vault-and-keys 24. TrustFix — Machine Identity Security Platform, https://trustfix.dev/ 25. Security architecture · Waydock, https://waydock.ai/docs/security-architecture 26. Building a Secrets Management Platform: Encryption, Key Hierarchy, https://medium.com/codetodeploy/building-a-secrets-management-platform-encryption-key-hierarchy-and-access-control-43a85bb38177